

Lösungsblatt: Eigenschaften verteilter Systeme

Modul 321 – Lektion 2

Hinweis: Dieses Lösungsblatt enthält Musterlösungen und einen möglichen Erwartungshorizont. Andere Antworten sind korrekt, wenn sie fachlich stimmig begründet sind und die Eigenschaften verteilter Systeme sauber einordnen.

1. Begriffe sauber erklären

Begriff	Mögliche Lösung
Idempotenz	Eine Operation ist idempotent, wenn dieselbe Anfrage mehrfach ausgeführt werden kann, ohne die fachliche Wirkung mehrfach anzuwenden. Mehrere gleiche Aufrufe führen also zum gleichen Endzustand wie ein einziger Aufruf.
Retry	Retry bedeutet, dass eine fehlgeschlagene oder unklare Anfrage nochmals gesendet wird. Das ist vor allem bei temporären Fehlern sinnvoll, muss aber begrenzt und fachlich abgesichert sein.
Resilience / Design for Failure	Resilience bedeutet, dass ein System auch bei Teilfehlern kontrolliert brauchbar bleibt. Design for Failure heisst, dass Ausfälle, Timeouts und Überlast von Anfang an als normale Fälle mitgeplant werden.
CQRS	CQRS trennt Schreiboperationen und Leseoperationen in unterschiedliche Modelle oder Komponenten. Dadurch können beide Seiten gezielt für ihre jeweilige Aufgabe optimiert werden.
Event Sourcing	Beim Event Sourcing werden fachliche Änderungen als Ereignisse gespeichert, nicht nur der aktuelle Zustand. Der Zustand kann später aus der Reihenfolge dieser Events wieder aufgebaut werden.

2. Retry beurteilen

- Timeout durch temporären Netzwerkfehler: Ja. Ein Retry ist sinnvoll, wenn Backoff, Limits und möglichst Idempotenz berücksichtigt werden.
- Ungültige Kundendaten: Nein. Der gleiche Request wird erneut fachlich scheitern.
- Zahlung möglicherweise bereits verarbeitet: Nur unter Bedingungen. Ohne Idempotenz besteht das Risiko einer doppelten Belastung.
- HTTP 503 wegen Überlastung: Ja, vorsichtig. Retries mit Exponential Backoff und Jitter können sinnvoll sein, dürfen den Partnerdienst aber nicht zusätzlich überlasten.

3. Idempotenz im Praxisfall

1. Es droht eine doppelte Zahlung oder eine doppelte fachliche Buchung.
2. Ein einfacher Retry kann dieselbe nicht-idempotente Aktion nochmals auslösen, obwohl der erste Versuch intern bereits erfolgreich war.
3. Der Client sendet pro fachlicher Operation einen eindeutigen Idempotency Key mit. Der Payment-Service merkt sich, dass diese Operation schon verarbeitet wurde, und liefert bei einem erneuten Request mit gleichem Schlüssel dasselbe Ergebnis zurück.
4. Mindestens Schlüssel, fachliche Identität der Operation und das Ergebnis bzw. den Verarbeitungsstatus der ersten Ausführung.

4. Resilience auf Architektur-Ebene

1. Schlechtes Verhalten: Die Produktseite lädt gar nicht mehr oder der Checkout wird blockiert, nur weil Empfehlungen fehlen.
2. Resilientes Verhalten: Produktkatalog, Warenkorb und Kaufprozess funktionieren weiter; nur der Empfehlungsbereich zeigt Fallback-Inhalte oder bleibt leer.
3. Mögliche Massnahmen: Timeouts, Circuit Breaker, Fallbacks, asynchrone Kommunikation, Caching, getrennte Ressourcen bzw. Isolation einzelner Dienste.
4. Degradierter Betrieb ist besser, weil die Kernfunktion für Benutzerinnen und Benutzer verfügbar bleibt und nicht das ganze Geschäft wegen eines Nebendienstes stillsteht.

5. Transaktionsgrenzen verstehen

1. Jeder Service besitzt typischerweise seine eigene Datenhaltung und läuft unabhängig. Eine einzige klassische ACID-Transaktion über alle Grenzen hinweg würde starke Kopplung, Koordinationsaufwand und betriebliche Risiken erzeugen.
2. Langsame oder ausgefallene Services können den Prozess blockieren, zu Timeouts führen oder Zwischenschritte in uneinheitlichen Zuständen hinterlassen.
3. Eventual Consistency bedeutet hier, dass nicht sofort überall derselbe Endzustand sichtbar sein muss, sondern dass sich das Gesamtsystem nach kurzer Zeit synchronisiert.
4. Beispiel: Wenn Zahlung fehlschlägt, wird die Bestellung storniert oder die Bestandsreservation wieder freigegeben.

6. Event Sourcing einordnen

1. Passt nicht. Das wäre eher ein klassisches Zustandsmodell ohne Historie.
2. Passt. Genau solche fachlichen Ereignisse werden typischerweise dauerhaft gespeichert.
3. Passt. Read-Modelle oder Projektionen können aus der Event-Folge neu aufgebaut werden.
4. Passt nicht. Für einfache CRUD-Anwendungen ist Event Sourcing oft unnötig komplex.

7. Entwurfsaufgabe

Mögliche Architektur:

- Frontend für Lernende und Mitarbeitende
- Reservation-Service
- Datenbank für Reservierungen und Arbeitsplätze
- Read-Modell oder Verfügbarkeitsservice
- E-Mail-Service oder externer Benachrichtigungsdienst

Mögliche Antworten:

1. Die Architektur kann aus Frontend, API/Reservation-Service, Datenbank und Mail-Service bestehen; optional zusätzlich ein separates Read-Modell für Verfügbarkeiten.
2. Retry ist etwa beim Versand einer Bestätigungs-E-Mail oder bei einem Aufruf eines externen Mail-Providers sinnvoll, wenn ein temporärer Netzwerkfehler auftritt.
3. Idempotenz ist wichtig bei der Erstellung einer Reservation, damit ein doppelter Klick oder ein verlorenes Response-Paket nicht zu zwei identischen Buchungen führt.
4. Degradierter Betrieb: Neue Reservationen funktionieren weiter, aber E-Mails werden später nachgesendet oder das Statistik-Dashboard ist vorübergehend veraltet.
5. CQRS könnte sinnvoll sein, wenn Verfügbarkeiten sehr häufig gelesen werden, während Reservationen vergleichsweise selten geschrieben werden. Event Sourcing wäre nur dann sinnvoll, wenn eine vollständige fachliche Historie oder ein Audit Trail explizit wichtig ist; sonst wäre es vermutlich unnötig komplex.

Didaktischer Hinweis

Die Muster der Lektion sollen nicht als Pflichtarchitektur verstanden werden. Entscheidend ist, dass Lernende erkennen, wann Retry, Idempotenz, Resilience, CQRS oder Event Sourcing einen klaren Mehrwert liefern und wann sie zusätzliche Komplexität ohne grossen Nutzen erzeugen.